

API seguras con JWT conectadas a Python y mostrarlas en Streamlit

ESTRUCTURA DEL PROYECTO

```
/Proyecto
├── Controllers/
│   └── AuthController.cs
├── Data/
│   └── AppDbContext.cs
├── Models/
│   ├── Usuexterno.cs
│   └── LoginRequest.cs
├── Services/
│   └── JwtService.cs
├── appsettings.json
└── Program.cs
```

Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.5

Install-Package Microsoft.EntityFrameworkCore.Tools -Version 8.0.5

Install-Package Microsoft.AspNetCore.Authentication.JwtBearer -Version 8.0.5

Install-Package System.IdentityModel.Tokens.Jwt

1. appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=TU_SERVIDOR;Database=TU_DB;User
    Id=usuario;Password=clave;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "CLAVE_SUPER_SECRETA_123456",
```

```
"Issuer": "miapi",  
"Audience": "miapiUsuarios"  
}  
}
```

2. Models/Usuexterno.cs

```
public class Usuexterno  
{  
    public int ideusuarioexterno { get; set; }  
    public string usuario { get; set; }  
}
```

3. Models/LoginRequest.cs

```
public class LoginRequest  
{  
    public string Usuario { get; set; }  
    public string Password { get; set; }  
}
```

4. Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;  
  
public class AppDbContext : DbContext  
{  
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }  
  
    public DbSet<Usuexterno> Usuexterno { get; set; }
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Usuexterno>().HasNoKey(); // porque usamos SQL directo
}
}
```

5. Services/JwtService.cs

```
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

public class JwtService
{
    private readonly IConfiguration _config;

    public JwtService(IConfiguration config)
    {
        _config = config;
    }

    public string GenerarToken(string usuario)
    {
        var claims = new[]
        {
            new Claim(ClaimTypes.Name, usuario)
        };

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_config["Jwt:Key"])
```

```

    );

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddHours(2),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
}

```

6. Controllers/AuthController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly JwtService _jwt;
}

```

```

public AuthController(AppDbContext context, JwtService jwt)
{
    _context = context;
    _jwt = jwt;
}

[HttpPost("login")]
public IActionResult Login([FromBody] LoginRequest login)
{
    var sql = @"
SELECT ideusuarioexterno, usuario
FROM usuexterno
WHERE usuario = @usuario
AND ideestado = 1
AND idetipousuarioexterno = 6
AND CAST(DECRYPTBYPASSPHRASE('INFOSRC2-JVDL', contraseña) AS VARCHAR(255)) =
@password
AND (fecvencimiento IS NULL OR fecvencimiento >= GETDATE())
";

    var user = _context.Usuexterno
        .FromSqlRaw(sql,
            new SqlParameter("@usuario", login.Usuario),
            new SqlParameter("@password", login.Password))
        .AsEnumerable()
        .FirstOrDefault();

    if (user == null)
        return Unauthorized("Usuario o contraseña incorrectos");

    var token = _jwt.GenerarToken(user.usuario);

```

```
        return Ok(new { token });  
    }  
}
```

7. Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;  
using Microsoft.EntityFrameworkCore;  
using Microsoft.IdentityModel.Tokens;  
using System.Text;  
  
var builder = WebApplication.CreateBuilder(args);  
  
// DB  
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));  
  
// JWT  
var key = builder.Configuration["Jwt:Key"];  
  
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
    .AddJwtBearer(options =>  
    {  
        options.TokenValidationParameters = new TokenValidationParameters  
        {  
            ValidateIssuer = true,  
            ValidateAudience = true,
```

```
        ValidateIssuerSigningKey = true,  
        ValidateLifetime = true,  
        ValidIssuer = builder.Configuration["Jwt:Issuer"],  
        ValidAudience = builder.Configuration["Jwt:Audience"],  
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(key))  
    };  
};
```

```
builder.Services.AddAuthorization();
```

```
// Servicios
```

```
builder.Services.AddScoped<JwtService>();
```

```
builder.Services.AddControllers();
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

8. Endpoint protegido (opcional)

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class DatosController : ControllerBase
{
    [Authorize]
    [HttpGet]
    public IActionResult Get()
    {
        return Ok("Acceso autorizado");
    }
}
```



```

[HttpPost("login")]
public IActionResult Login([FromBody] LoginRequest login)
{
    var sql = @"
        SELECT ideusuarioexterno, usuario
        FROM usuexterno
        WHERE usuario = @usuario
        AND ideestado = 1
        AND idetipousuarioexterno = 6
        AND CAST(DECRYPTBYPASSPHRASE('INFOSRC2-JVDL', contraseña) AS VARCHAR(255)) = @password
        AND (fecvencimiento IS NULL OR fecvencimiento >= GETDATE())
    ";

    var user = _context.Usuexterno
        .FromSqlRaw(sql,
            new SqlParameter("@usuario", login.Usuario),
            new SqlParameter("@password", login.Password))
        .AsEnumerable()
        .FirstOrDefault();

    if (user == null)
        return Unauthorized("Usuario, contraseña o estado inválido");

    var token = _jwt.GenerarToken(user.usuario);

    return Ok(new { token });
}

```

9. FUNCIÓN PARA CONSUMIR API

```
import requests

def login_api(usuario, password):
    url = "http://localhost:5000/api/auth/login"

    data = {
        "usuario": usuario,
        "password": password
    }

    response = requests.post(url, json=data)

    if response.status_code == 200:
        token = response.json()["token"]
        return token
    else:
        print("Error:", response.text)
        return None

# uso
token = login_api("usuario1", "123456")
print(token)
```

.NET (consumir API)

```
using System.Net.Http;
using System.Text;
using System.Text.Json;

public async Task<string> LoginApi()
{
    var client = new HttpClient();

    var data = new
    {
        usuario = "usuariol",
        password = "123456"
    };

    var content = new StringContent(
        JsonSerializer.Serialize(data),
        Encoding.UTF8,
        "application/json"
    );

    var response = await client.PostAsync("http://localhost:5000/api/auth/login", content);

    var result = await response.Content.ReadAsStringAsync();

    return result;
}
```

